Date: 03 / 06 / 2026

Duration: 3 hours

FINAL ASSESSMENT

III YEAR – SEMESTER 6

CSE23AE304 - PCP V

(B.Tech. E01,E02,E03,E04)

SET D

Dataset D – Leave & Attendance Workflow Management System

Assessment Objective

This assessment evaluates your ability to build a complete MERN Full Stack application involving:

- Authentication & Authorization
- External API Integration
- Data Validation & Sanitization
- MongoDB Persistence
- Relationship Modeling
- REST API Development
- Querying & Aggregation
- Frontend Integration
- Protected Routes
- State Management
- Deployment Readiness

The assessment focuses on engineering workflows rather than isolated CRUD operations.

Problem Statement

You are required to develop a:

Leave & Attendance Workflow Management System

The system manages:

- Employees
- Departments
- Attendance Records
- Leave Requests
- Approval Workflows

Students must:

1. Fetch attendance dataset from external API
2. Validate and sanitize records
3. Persist valid records into MongoDB Atlas
4. Build secure REST APIs
5. Implement role-based authorization
6. Build React frontend consuming backend APIs
7. Implement analytics & dashboard features

Standard Response Structure

Success Response

```
{
  "success": true,
  "message": "Operation successful",
  "data": []
}
```

Error Response

```
{
  "success": false,
  "message": "Error message"
}
```

Section A — Authentication & Authorization

Objective

Implement JWT-based authentication and role-based access control.

Q1 — Register API

POST /auth/register

Requirements

- Register new employee
- Password hashing mandatory
- Duplicate email prevention
- Store role

Roles

- admin
- hr_manager
- team_lead
- employee

Q2 — Login API

POST /auth/login

Requirements

- JWT token generation
- Invalid credential handling
- Return authenticated user details

Q3 — Current User API

GET /auth/me

Requirements

- Protected route

- Return logged-in user details

Authorization Rules

Only:

- admin
- hr_manager

can:

- approve/reject leaves
- access analytics APIs
- manage attendance policies
- manage departments

Team leads can:

- review team attendance
- recommend leave approvals

Employees can:

- apply for leave
- view own attendance
- view leave history

Section B — Dataset Synchronization & MongoDB Persistence

Objective

Fetch dataset from external API, validate records, sanitize values, and persist valid records into MongoDB Atlas.

Q4 — Sync API

POST /sync

Responsibilities

1. Fetch dataset from private API
2. Validate records
3. Normalize values
4. Reject invalid entries
5. Prevent duplicate insertion
6. Persist valid records into MongoDB
7. Return sync summary

Example Response

```
{
  "success": true,
  "totalFetched": 120,
  "inserted": 92,
  "duplicates": 11,
  "rejected": 17
}
```

Required Sanitization Examples

Raw Value	Expected Value
" PRESENT "	"present"
" Casual Leave "	"Casual Leave"
"hr"	"HR"

Validation Requirements

Reject:

- invalid attendance status
- invalid leave dates
- duplicate employee IDs
- invalid department references
- negative leave balances
- invalid approval status

Q5 — Health API

GET /health

Example

```
{
  "success": true,
  "database": "connected",
  "documentCount": 92
}
```

Section C — MongoDB Persistence, Relationships & Runtime Validation

Objective

Implement MongoDB persistence with proper relationships, validation logic, and runtime data consistency.

Required Collections & Relationships

Employee

Fields

- employeeId
- name
- email
- role
- department

- designation
- leaveBalance
- status

Department

Fields

- departmentId
- name
- manager
- employeeCount
- status

Attendance

Relationships

- employee → Employee

Fields

- attendanceId
- date
- checkIn
- checkOut
- status
- workingHours

Leave Request

Relationships

- employee → Employee
- approvedBy → Employee

Fields

- leaveId
- leaveType
- fromDate
- toDate
- reason
- approvalStatus

Activity Log

Relationships

- employee → Employee

Fields

- action
- previousStatus
- newStatus
- timestamp

Section D — CRUD & Workflow APIs

Objective

Implement REST APIs with proper validation and workflow handling.

Q6 — Employee APIs

GET /employees
GET /employees/:id

Q7 — Department APIs

POST /departments
GET /departments
GET /departments/:id
PATCH /departments/:id
DELETE /departments/:id

Q8 — Attendance APIs

POST /attendance
GET /attendance
GET /attendance/:id
PATCH /attendance/:id
DELETE /attendance/:id

Q9 — Leave APIs

POST /leaves
GET /leaves
GET /leaves/:id
PATCH /leaves/:id
DELETE /leaves/:id

Workflow Rules

- Employees cannot apply leave for past dates
- Leave requests exceeding balance must be rejected
- Attendance cannot be marked twice for same date
- Inactive employees cannot apply for leave
- Approved leave dates should not allow attendance marking

Section E — Leave Approval Workflow APIs

Objective

Implement protected workflow APIs.

Date: 03 / 06 / 2026

Duration: 3 hours

FINAL ASSESSMENT

III YEAR – SEMESTER 6

CSE23AE304 - PCP V

(B.Tech. E01,E02,E03,E04)

SET D

Q10 — Approve Leave Request

PATCH /leaves/:id/approve

Requirements

- Only admin/hr_manager allowed
- Leave request must exist
- Approved leaves cannot be reapproved

Q11 — Update Leave Status

PATCH /leaves/:id/status

Allowed Status

- pending
- approved
- rejected
- cancelled

Workflow Rules

- Rejected leaves cannot move directly to approved
- Cancelled leaves cannot be approved
- Employees cannot approve their own leave
- Approved leave should reduce leave balance

Section F — Filtering, Search & Pagination

Objective

Implement scalable querying APIs.

Q12 — Attendance Filters

Examples

GET /attendance?status=present
GET /attendance?department=HR
GET /attendance?date=2026-05-01

Q13 — Leave Filters

Examples

GET /leaves?approvalStatus=approved
GET /leaves?leaveType=Casual

Q14 — Pagination & Search

Examples

GET /employees?page=1&limit=10
GET /employees?search=dinesh

Combined Query Example

GET /leaves?
approvalStatus=pending&
leaveType=Sick&
page=2&
limit=5

Section G — Analytics & Aggregation APIs

Objective

Implement MongoDB aggregation APIs.

Q15 — Attendance Analytics

GET /analytics/attendance

Return

- total employees
- present employees
- absent employees
- attendance percentage

Q16 — Leave Analytics

GET /analytics/leaves

Return

- total leave requests
- approved leaves
- rejected leaves
- pending requests

Q17 — Department Analytics

GET /analytics/departments

Return

- department-wise attendance percentage
- highest leave requests
- active employee count
- leave utilization percentage

Section H — Frontend Integration

Objective

Build React frontend consuming backend APIs.

Date: 03 / 06 / 2026

Duration: 3 hours

FINAL ASSESSMENT

III YEAR – SEMESTER 6

CSE23AE304 - PCP V

(B.Tech. E01,E02,E03,E04)

SET D

Mandatory Frontend Features

Authentication

- Login page
- JWT token handling
- Protected routes
- Role-based rendering

Dashboard

Display:

- attendance analytics cards
- leave summaries
- pending approvals
- recent attendance records

Employee Module

- apply leave
- view attendance
- track leave history
- filter attendance

HR Manager Module

- approve/reject leave
- manage departments
- view analytics
- monitor attendance

Mandatory State Management

Use:

- Context API
- Reducer

Section I — Frontend Evaluator Compliance, Global State & Protected Routes (5 Marks)

Objective

Evaluate frontend architecture.

Requirements

Implement:

- Protected routes
- Role-based navigation
- Reducer logic
- Global state management
- Required data-testid attributes
- Dynamic UI updates based on global state

Mandatory Global State

Your application must maintain global state for:

- authenticated user
- JWT token
- employees
- departments
- attendance
- leaves
- filters/search values
- analytics/dashboard data

Use:

- Context API
- Reducer

Mandatory data-testid Attributes

Authentication

Component	data-testid
Login form	login-form
Email input	email-input
Password input	password-input
Login button	login-btn
Logout button	logout-btn

Navigation

Component	data-testid
Navbar	navbar
Dashboard link	dashboard-link
Employees link	employees-link
Departments link	departments-link
Attendance link	attendance-link
Leaves link	leaves-link

Dashboard

Component	data-testid
Analytics container	analytics-container
Total employees card	total-employees-card
Attendance percentage card	attendance-percentage-card

Pending leaves card	pending-leaves-card
Leave analytics chart	leave-chart
Recent attendance section	recent-attendance

Attendance Module

Component	data-testid
Attendance table	attendance-table
Attendance search input	attendance-search
Attendance filter dropdown	attendance-filter
Attendance row	attendance-row

Leave Module

Component	data-testid
Leave table	leave-table
Apply leave button	apply-leave-btn
Approve leave button	approve-leave-btn
Leave search input	leave-search
Pagination next button	pagination-next
Pagination previous button	pagination-prev

Department Module

Component	data-testid
Department table	department-table
Create department button	create-department-btn
Department row	department-row

Section J — Frontend Evaluator Compliance & Deployment (10 Marks)

Objective

Validate deployment readiness, frontend evaluator compatibility, and runtime UI behavior using automated Playwright-based evaluation.

Mandatory Frontend Behavior

Your frontend must:

- consume your own backend APIs
- update UI dynamically after CRUD operations
- maintain JWT token in global state
- maintain authenticated session after refresh
- restrict unauthorized route access
- render UI based on user role

Mandatory React Router Routes

Students must implement the following routes using React Router DOM.

Route	Purpose
/login	Authentication
/dashboard	Analytics dashboard
/employees	Employee management
/departments	Department management
/attendance	Attendance management
/leaves	Leave management
/profile	Current user profile

Global State Exposure Requirement

To support automated Playwright evaluation, students must expose global application state using:

window.appState

The global state object must dynamically reflect the current application state.

Mandatory window.appState Structure

```
window.appState = {
  authUser,
  token,
  employees,
  departments,
  attendance,
  leaves,
  filters,
  analytics
}
```